Computational Tools for Modelling and Analysis: Dimensionality Reduction

Dr Iain Styles

November 6, 2015

1 *Introduction*

Many types of data consist of measurements of multiple quantities at each sampling point.

- Fluorescence imaging at multiple wavelengths.
- Mass spectrometry data.
- DNA Microarray data.

Typically these datasets can have hundreds to millions of samples, of tens to hundreds of thousands of variables, and we will ultimately want to apply some type of pattern recognition method to identify structure and patterns in the data. It is too complex to visualise such data really effectively and we must apply some kind of pattern recognition such as clustering to understand what the data is telling us, but there are two challenging cases:

- Large numbers of samples. This imposes additional computational demands that can in principle be overcome with more resource. This is not always the case as some methods do not scale nicely with the size of the problem,
- Large numbers of variables. This also imposes a computational burden, but not one that can be so easily overcome by additional effort. There is also a much more insidious issue that must be addressed.

In this lecture, we will focus on the second of these issues: how to analyse data of very large "dimensionality", by which we typically mean hundreds to thousands of measurements per sample.

2 *The Curse of Dimensionality*

Measurements consisting of many variables pose some serious challenges with regard to the application of computation data analysis methods. There are many problems:

- The data is hard to visualise. How can you "picture" data in more than three dimensions? There are some "tricks" involving colours and shapes you can use to visualise 6–7 dimensions. You can also use so-called spider diagrams, but it becomes increasingly more difficult as the dimensionality of the data increases.
- There is, as with the case of many samples, potential for considerable computational cost.
- Most importantly, our geometrical intuition can break down in high-dimensional spaces. Consider two spheres, of radius r and $r - \delta$ respectively (with $\delta \ll r$). The volume of a sphere (circle, hypersphere) scales as r^d, where d is the dimensionality of the space, and so in the high-dimensional limit, the ratio of the volume

of a sphere to the volume of its outer shell is

$$\lim_{d\to\infty} \frac{r^d - (r-\delta)^d}{r^d} = 1 \tag{1}$$

Thus most of the volume of the hypersphere is concentrated in a thin shell around its edge and most of the volume is at the edges of the space. A similar phenomenon can be seen in the behaviour of distances in large numbers of dimensions: considering the minimum and maximum distances between pairs of points it can be proven that

$$\lim_{d\to\infty} E\left(\frac{\mathrm{dist}_{\max}(d) - \mathrm{dist}_{\min}(d)}{\mathrm{dist}_{\min}(d)}\right) \to 0 \tag{2}$$

Thus, the concept of distance in high-dimensional spaces is rather less useful than in the low-dimensional spaces we are accustomed to. This has serious implications for any algorithm that uses the distances between points in some way (e.g. k-means).

One notable feature of high-dimensional data is that there is frequently "redundancy" in the variables. For example, two variables may be very strongly correlated and the data actually lie on a "line". In this case, it is often possible to find an alternative coordinate system for the data that removes this redundancy. This is illustrated in Figure. 1.

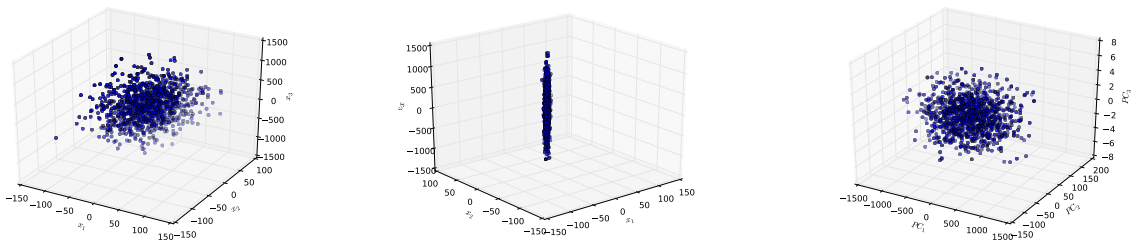

Figure 1: Three views of a three-dimensional dataset. Left: view in original coordinate system. Centre: View in original coordinate system from viewpoint that reveals the two-dimensional nature of the data. Right: dataset viewed in coordinate system aligned with the data. Note the scales on the axes.

3 *Principal Component Analysis*

The most common technique for finding a transformation that removes redundant dimensions in a dataset is **principal component analysis**. The example in Figure 1 is of a fundamentally two-dimensional dataset embedded in a three-dimensional space, and we want to find a coordinate system that is a natural description of the object – in this case a two-dimensional coordinate system. There are infinitely many possible choices of such a coordinate system and we have to determine a set of criteria by which we will choose the appropriate one. In PCA, we choose a coordinate system such that:

- The first coordinate is aligned with the direction of greatest variation in the data.

- The second coordinate is perpendicular to the first coordinate and aligned with the direction of greatest variation under that constraint,
- Subsequent coordinates are perpendicular to all that have already been defined, aligned with the direction of greatest variation under that constraint.

We begin by computing the covariance matrix, $\mathbf{\Sigma}$, which describes the way different variables "co-vary'. The components of $\mathbf{\Sigma}$ are

$$\Sigma_{ij} = \frac{1}{N-1} \sum_{n=1}^{N} \left(x_i^{(n)} - \bar{x}_i \right) \left(x_j^{(n)} - \bar{x}_j \right) \tag{3}$$

and they describe the covariance between the variables x_i and $x - j$, measured over N samples. $\bar{x}_i$ is the mean value of x_i.

There is a simple way to compute $\mathbf{\Sigma}$ if we organise our dataset in a sensible way. First, we construct the matrix $\mathbf{X}$ with components $X_{ij} = x_i^{(j)}$; that is, each *row* of $\mathbf{X}$ corresponds to a variable (eg. a wavelength), and each *column* corresponds to a sample (eg. a pixel). Then, compute the mean of every row (variable) and tile the resulting column vector side-by-side N times such that every column is identical and every row contains the mean value of the corresponding variable in every column. We will call the new matrix $\bar{\mathbf{X}}$. Then, we form the covariance matrix as

$$\mathbf{\Sigma} = \frac{1}{N-1} \left(\mathbf{X} - \bar{\mathbf{X}} \right) \left(\mathbf{X} - \bar{\mathbf{X}} \right)^{\mathrm{T}} = \tilde{\mathbf{X}} \tilde{\mathbf{X}}^{\mathrm{T}} \tag{4}$$

Now we have a measure of the variance in the dataset, we need to find the natural set of vectors that describe this data. To this end, we start by finding the direction of greatest variance. Let us define a new matrix $\mathbf{T} = \begin{pmatrix} \mathbf{t}_1 & \mathbf{t}_2 & \ldots \mathbf{t}_M \end{pmatrix}$ such that the columns of $\mathbf{T}$ (the vectors $\mathbf{t}$) represent a set of new orthogonal (perpendicular) directions in the space and collectively form a new coordinate system (basis) for the space. We will take these vectors to be normalised such that $\mathbf{T}^{\mathrm{T}}\mathbf{T} = \mathbf{I} \leftrightarrow \mathbf{t}_i \cdot \mathbf{t}_j = \delta_{ij}$. In the first instance, we wish to find the vector $\mathbf{t}^*$ such that the variance of the projection is maximised, that is:

$$\mathbf{t}_1 = \arg\max_{\mathbf{t}} \operatorname{var}(\mathbf{t}^{\mathrm{T}}\mathbf{X}) \tag{5}$$

In this new direction, the variance of the data is given by $\mathbf{t}_1^{\mathrm{T}}\tilde{\mathbf{X}}\tilde{\mathbf{X}}^{\mathrm{T}}\mathbf{t}_1 = \mathbf{t}_1^{\mathrm{T}}\mathbf{\Sigma}\mathbf{t}_1$ (ignoring the pre-factor $1/(N-1)$) and hence our goal is to find $\mathbf{t} = bt_1$ that maximises this. We must also impose the normalisation constraint $\mathbf{t}^{\mathrm{T}}\mathbf{t} = 1$ in order to find a normalised vector. We use a mathematical "trick" known as a Lagrange multiplier to acheive this. The quantity we wish to maximise is $\mathbf{t}^{\mathrm{T}}\mathbf{\Sigma}\mathbf{t}$, from which we subtract a "multiple of zero":

$$\mathbf{t}_1 = \arg\max_{\mathbf{t}} \left[\mathbf{t}^{\mathrm{T}}\mathbf{\Sigma}\mathbf{t} - \lambda_1 (\mathbf{t}^{\mathrm{T}}\mathbf{t} - 1) \right] \tag{6}$$

We now differentiate this with respect to λ and set to zero to maximise, and we find that $\mathbf{t}_1$ should satisfy

$$\left(\mathbf{\Sigma} - \lambda_1 \mathbf{I} \right) \mathbf{t}_1 = 0 \tag{7}$$

or equivalently

$$\mathbf{\Sigma t}_1 = \lambda_1 \mathbf{t}_1 \tag{8}$$

This is an *eigenvalue* problem: λ_1 and $\mathbf{t}_1$ are an eigenvalue/vector pair of $\mathbf{\Sigma}$. Since our aim is to maximise variance, and given that along $\mathbf{t}_1$, the variance is $\mathbf{t}_1^{\mathrm{T}}\mathbf{\Sigma t}_1 = \lambda_1 \mathbf{t}_1^{\mathrm{T}}\mathbf{t}_1 = \lambda_1$. In plain English, the greatest variance in the data is given by its largest eigenvalue, and the direction of greatest variance is the accompanying eigenvector. Some further simple but algebraically involved calculations can be done to show that the next-greatest direction of variation is given by the second-largest eigenvalue/vector pair, and so on. In summary:

The principal components of a dataset are the eigenvectors of the covariance metric, ordered by eigenvalue.

3.1 *Calculating the Principal Components*

Once it is understood what calculation has to be done, it is computationally straightforward to perform PCA: find the eigenvalues/vectors of the covariance matrix. Given $\mathbf{T} = \begin{pmatrix} \mathbf{t}_1 & \mathbf{t}_2 & \mathbf{t}_n \end{pmatrix}$ and $\mathbf{S} = \mathrm{diag}\left(\lambda_1, \lambda_2, \ldots \lambda_n\right)$ then

$$\mathbf{\Sigma} = \mathbf{TST}^{\mathrm{T}} \tag{9}$$

and the transformation of the data into the new coordinate system is given by

$$\mathbf{X}' = \mathbf{T}^{\mathrm{T}}\tilde{\mathbf{X}} \tag{10}$$

where we have elected to use the zero-mean data. The columns of $\mathbf{X}'$ are known as the **scores**: each corresponds to the data projected onto the corresponding principal component axis, with the variance in each of the scores given by the corresponding entry on the diagonal of $\mathbf{S}$.

One of the goals of PCA is to allow us to remove non-varying dimensions. This is easily acheived. We will find it useful to consider the "total variance" of the dataset, given by the sum of the eigenvalues: $V_{\mathrm{T}} = \sum_i S_{ii}$. It is then possible to calculate what proportion of the total variance is contributed along each of the principal component axes by computing $\lambda_i / V_{\mathrm{T}}$. This allows us to determine how many of the principal components are necessary to describe the majority of the variance in the data. Typically, and especially with very high dimensional data, the amount of variance in each successive component decreases rapidly and when the cut-off has been determined, one can simply select the "significant dimensions" by truncating the columns of $\mathbf{T}$ to remove the non-varying directions.

The application of PCA to a dataset will thus typically follow the following workflow:

1. Form the data matrix $\mathbf{X}$.
2. Form the zero-mean matrix $\tilde{\mathbf{X}}$.
3. Calculate the covariance matrix $\mathbf{\Sigma}$
4. Calculate the eigenvalues/vectors of $\mathbf{\Sigma}$

5. Sort the eigenvalues/eigenvectors from largest to smallest by eigenvalue.

6. Choose a proportion of variance to retain and discard the eigenvalues/vectors for non-varying dimensions.

7. Compute the scores by projecting the data onto the new coordinates.

PCA for dimensionality reduction is often done as part of the pre-processing workflow before the data is analysed. As it is a purely linear transformation, the relationship between the points in the data is preserved, but since the dimensionality is reduced, the distances now become more meaningful and discriminating between data points. Thus, it is very common to use PCA to reduce data size and dimensionality prior to the application of clustering or pattern recognition techniques to find structure within the data.

4 *Data Normalisation and Variable Scaling*

In some multivariate datasets, where the variables are of a different type or on a different scale, it is sometimes useful to consider mapping all variables onto a common scale. For example, if we are measuring two positively correlated variables of equal variance then the two PCs will be at 45°to the variable axes, but if we rescale one variable by a factor of 100, the first PC will align almost exactly with that variables axis. In such cases it is common to rescale all variables to have unit variance. One should consider carefully how to use PCA in such a situation.

5 *Random Projection*

One problem with PCA is that the construction and manipulation of the covariance matrix is computationally very expensive, and for very large number of variables it will not be possible to do this calculation in a "normal" computer. An increasingly popular alternative which seems deceptively simple but is fundamentally very powerful is the random projection. In this method, rather than choosing the vectors onto which the data will be projected according to some underlying principle that determines the property of the projection, we instead project onto a series of random vectors, representing a series of random directions in space. Mathematically, Given a dataset $\mathbf{X}_{m\times n}$ of m variable from n samples, form a random matrix $\mathbf{R}_{m\times k}$ where each column is a normalised random vector. That is:

$$\mathbf{R} = \begin{pmatrix} \mathbf{r}^{(1)} & \mathbf{r}^{(2)} & \dots & \mathbf{r}^{(k)} \end{pmatrix} \tag{11}$$

where ${\mathbf{r}^{(i)}}^{\mathrm{T}}\mathbf{r}^{(i)} = 1$. Then the dataset is projected onto these new vectors by computing

$$\mathbf{X}' = \mathbf{R}^{\mathrm{T}}\mathbf{X}. \tag{12}$$

These are analagous to the scores we computed in PCA. Unlike PCA, there is no natural metric to decide how many projections are

necessary and so one needs to use one's judgement as to how many may be required.

Random projections have been shown to be surprisingly effective for dimensionality reduction. The key mathematical results underpinning its effectiveness is the Johnson-Lindenstrauss lemma, which exploit a phenomenon known as the "concentration of measure". A full explanation of these results is beyond the scope of this course. They have found popularity for several important reasons.

- They are very inexpensive to compute.
- For very high-dimensional data, randon vectors are very likely to be orthogonal.
- It can be shown that they presereve certain important distance metricss.

6 *More Sophisticated Alternatives*

Both PCA and random projections are purely linear transformations of the data. They do not change the "shape" of a cloud of datapoints, but simply project it onto a set of globally linear coordinates. Whilst this can be effective for finding and removing globally non-varying directions in a dataset, it is not suited for finding finer structure in a dataset. Some examples of non-linear dimensionality reduction techniques include

- Self-organising maps
- Curvilinear component analysis
- Latent variable models
- t-SNE (t-distributed stochastic neighbour embedding)

amongst many others. A common use of the linear methods is as pre-processing to eliminate globally redundant dimensions followed by subsequent pattern analysis using a non-linear method of a clustering technique. This is often required due to the expense of the more sophisticated methods on large dataset, so they often require a cruder reduction to be applied to make the data tractable.